

6 Documenting Software

Documentation is an indispensable but often unloved part of software development. The documentation of an application can be divided into the following different levels. It differs in its target audience and its proximity to the source code.

- **User manual**

This type of documentation is intended for the end users and support staff of the software and is therefore also relatively far removed from the source code of the application. A user manual is used to explain the features of the application, describe how the application can be installed if necessary, what problems frequently occur, and how they can be solved.

- **Technical specification**

The technical specification is aimed at developers, architects, and technical project managers. It has no direct reference to the code, but describes the architecture, components, and interfaces of the project. Data models, algorithms, and technologies used in the application are also included. The specification provides a good overview of the application and the interrelationships in the project.

- **API documentation**

The application programming interface (API) documentation depends on the type of software you create. If you work on a library, this documentation usually describes the public functions developers use when integrating the library. In the case of a web service, the API documentation describes the interfaces your service makes available to the outside world. The API documentation is always intended for developers who want to connect to your software and is therefore directly related to a part of your source code.

- **Class and function documentation**

This type of documentation is a direct component of your source code

and is aimed at developers who work directly with the code base and extend and maintain it. In the case of classes, the comments describe the purpose and everything worth knowing about the class. Function documentation describes the purpose of a function, the parameters, and the return value, as well as relevant implementation details.

- **Inline comments**

Inline comments have the most direct reference to the source code of all documentation variants; they are embedded in the code, either as separate lines or even as part of a program line. They are used to provide developers with direct information about the code.

In this chapter, we focus on the documentation types with a direct reference to the source code, that is, API documentation, class and function documentation, and inline comments. These types of documentation are mainly used to communicate information between members of the development team. Opinions differ widely as to which parts of the source code should be documented and how.

The different camps range from “every block in the code must be documented” to “my code is self-explanatory, so documentation is unnecessary.” We want to choose a pragmatic middle way: all public interfaces that other developers have to deal with should be documented. In addition, complex code passages should be provided with explanatory comments.

6.1 Challenges with Documentation

Documentation, like automated testing, isn’t necessarily a favorite discipline of developers. As with testing, there are also a number of challenges with documentation that you can overcome with the help of AI support. These include the following:

- **Time required**

Good documentation takes time. In function documentation, for example, it’s not enough to simply repeat the function name and list the names of the parameters. AI tools can do the work for you here by generating the documentation for you.

- **Continuous updating**

There are few things worse than outdated documentation. Many devel-

opers rely on the documentation in the code. If that isn't up-to-date, avoidable errors can occur. For this reason, you must also check the documentation every time you change the code and adapt it if necessary. You can also outsource this task, at least in part, to the AI tool.

- **Consistency and accuracy**

If different people work on the source code, inconsistencies can creep in. Everyone has their own way of documenting; if it differs too much from that of others, it can be distracting. With the right prompts and specific templates, an AI tool can ensure that the documentation looks as if it has been cast from a single mold.

- **Comprehensibility**

The people who work with the source code must understand the documentation. It should therefore be written in clear and precise language. Sometimes, complex concepts need to be explained as clearly as possible. Another complicating factor is that the documentation is usually written in English, but not everyone involved is a native speaker. AI can provide support here, formulate the documentation, and help with translation.